A Metamorphic Testing Perspective on Knowledge Distillation for Language Models of Code: Does the Student Deeply Mimic the Teacher?

Md. Abdul Awal[a,*], Mrigank Rochan[a], Chanchal K. Roy[a]

[a]*Department of Computer Science, University of Saskatchewan, 110 Science Pl, Saskatoon, S7N 5C9, Saskatchewan, Canada*

arXiv:2511.05476v1 [cs.SE] 7 Nov 2025

Abstract

Transformer-based language models of code have achieved state-of-the-art performance across a wide range of software analytics tasks, but their practical deployment remains limited due to high computational costs, slow inference speeds, and significant environmental impact. To address these challenges, recent research has increasingly explored knowledge distillation as a method for compressing a large language model of code (the teacher) into a smaller model (the student) while maintaining performance. However, the degree to which a student model deeply mimics the predictive behavior and internal representations of its teacher remains largely unexplored, as current accuracy-based evaluation provides only a surface-level view of model quality and often fails to capture more profound discrepancies in *behavioral fidelity* between the teacher and student models. To address this gap, we empirically show that the student model often fails to deeply mimic the teacher model, resulting in up to **285%** greater performance drop under adversarial attacks, which is not captured by traditional accuracy-based evaluation. In addition, to capture discrepancies in behavioral fidelity, we propose *MetaCompress*, a metamorphic testing framework that systematically evaluates behavioral fidelity by comparing the outputs of teacher and student models under a set of behavior-preserving metamorphic relations. We evaluate *MetaCompress* on two widely studied tasks, clone detection and vulnerability prediction, using compressed versions of popular language models of code, *CodeBERT* and *GraphCodeBERT*, obtained via three different knowledge distillation techniques: *Compressor*, *AVATAR*, and *MORPH*. The results show that *MetaCompress* identifies up to **62%** behavioral discrepancies in student models, underscoring the need for behavioral fidelity evaluation within the knowledge distillation pipeline and establishing *MetaCompress* as a practical framework for testing compressed language models of code derived through knowledge distillation.

Keywords: Large Language Models of Code, Knowledge Distillation, Adversarial Attack, Metamorphic Testing, Behavioral Fidelity

1. Introduction

Transformer-based [1] language models of code have achieved state-of-the-art performance across a wide range of software analytics tasks, including clone detection [2], code

*Corresponding author

Email addresses: abdul.awal@usask.ca (Md. Abdul Awal), mrochan@cs.usask.ca (Mrigank Rochan), chanchal.roy@usask.ca (Chanchal K. Roy)

summarization [3], vulnerability prediction [4], and code search [5]. However, despite their impressive capabilities, the adoption of these models remains limited, particularly in consumer-grade devices (e.g., laptops) and latency-sensitive applications, due to their substantial computational demands, slow inference speed, and considerable energy consumption and carbon footprint [6, 7]. For instance, CodeBERT, a widely used code language model with 125 million parameters and a size of 476 MB, can incur up to 1.5 seconds of latency, consume 0.32 kWh of energy, and emit 0.14 kg of CO_2 on consumer-grade laptops.[1] To address these practical challenges, the software engineering research community has recently begun exploring knowledge distillation methods [9] to develop extremely compact models that reduce model size, lower inference latency, and mitigate environmental impact while maintaining task performance [6, 10–15].

Knowledge distillation (KD) transfers knowledge from a large teacher model to a smaller student[2] model, maintaining comparable accuracy while requiring significantly fewer computational resources. In this context, Shi et al. [10] introduced *Compressor*, a genetic algorithm-based approach that compresses code-specialized language models (e.g., CodeBERT [16]) from 476 MB and 125 million parameters to only 3 MB with minimal loss in accuracy. Although KD effectively reduces computational costs and environmental impact, student model performance is typically evaluated using traditional accuracy-based metrics that assess whether the student model's task performance on ground-truth labels remains comparable to that of the teacher model. However, this form of evaluation offers only a surface-level view and may overlook critical discrepancies in **behavioral fidelity**—*the extent to which a student model deeply mimics the predictive behavior and internal representations of its teacher counterpart.*

We investigate the student model's ability to mimic the teacher by performing simple yet effective adversarial attacks [17–19] that rename identifiers based on the contextual semantics of each code snippet. The empirical results reveal that although the student models achieve performance comparable to the teacher before adversarial perturbations, their performance degrades substantially under adversarial attacks, with up to **285%** greater performance loss than the teacher model. This finding indicates that the student model does not deeply mimic the teacher's predictive behavior and internal representations, highlighting the limitations of the traditional accuracy-based evaluation, which often overlooks more profound behavioral discrepancies between teacher and student models. It also underscores the need for a nuanced evaluation framework that goes beyond accuracy to assess how deeply a student model mimics the predictive behavior and internal representations of its teacher, providing a more comprehensive understanding of model alignment under knowledge distillation.

To fill this gap, we introduce *MetaCompress*, a testing framework inspired by metamorphic testing principles in software engineering [20], to evaluate the behavioral fidelity of the student model in knowledge distillation. In contrast to traditional accuracy-based evaluation, which compares model performance against ground-truth labels, *MetaCompress* performs a head-to-head comparison of student and teacher model outputs across a suite of behavior-preserving metamorphic relations, enabling a more nuanced assessment of behavioral fidelity. We design four metamorphic relations, grounded in strong empir-

[1]When integrated into IDEs, CodeBERT may be invoked thousands of times daily by a developer, reflecting typical usage patterns [8].

[2]In this study, the term student model refers to the compressed model obtained through knowledge distillation, while the term teacher model denotes the original uncompressed model.

ical evidence from manual investigations, that capture discrepancies in the behavioral fidelity of student models, ranging from prediction-level disagreement to divergences in the probability distribution.

We empirically evaluate *MetaCompress* on two widely studied software engineering tasks, *clone detection* and *vulnerability prediction*, using two popular language models of code, *CodeBERT* [16] and *GraphCodeBERT* [21]. To distill their smaller student models, we apply three state-of-the-art knowledge distillation techniques designed for language models of code: *Compressor* [6], *AVATAR* [10], and *MORPH* [22]. Our extensive experiments show that *MetaCompress* can reveal behavioral fidelity discrepancies of up to **62%** across four metamorphic relations that traditional accuracy-based metrics fail to detect.

In summary, the main contributions of this paper are as follows:

- **Insight:** We are the first to demonstrate that traditional accuracy-based metrics fall short in capturing behavioral fidelity between student and teacher models by measuring adversarial robustness. To address this, we reframe this evaluation challenge as a classical software testing problem and address it through the lens of metamorphic testing. Ensuring behavioral fidelity is critical because it guarantees that the student model not only provides correct outputs but also behaves consistently across scenarios, just like the teacher model, which is essential for complex tasks like coding.
- **Technique:** We propose *MetaCompress*, a novel output-based metamorphic testing framework grounded in four behavior-preserving metamorphic relations to systematically evaluate the behavioral fidelity of student models for code
- **Evaluation:** We conduct a comprehensive evaluation of *MetaCompress*, demonstrating its effectiveness in uncovering discrepancies in the behavioral fidelity of student models for code that are often overlooked by traditional accuracy-based metrics.
- **Open science:** Our replication package has been open-sourced for transparency and reproducibility[3].

2. Preliminaries

2.1. Language Models of Code

Transformer-based [1] language models of code have demonstrated remarkable performance across a wide range of software analytics tasks, such as vulnerability prediction, clone detection, code summarization, code completion, code comment generation, and code search [16, 21, 23–30]. These models learn general language representations from large amounts of unlabeled data through a two-step process: (i) pre-training and (ii) fine-tuning. During pre-training, the model employs a self-supervised objective to acquire a general understanding of source code structures and patterns. Once the training is complete, the model can be fine-tuned by updating a few parameters for specific tasks with smaller datasets, making it more efficient in terms of time and computational resources compared to training a model from scratch [26].

Language models for code can be broadly classified into three categories: (1) encoder-only, (2) decoder-only, and (3) encoder-decoder models [19]. Encoder-only models (e.g., CodeBERT [16] and GraphCodeBERT [21]) process the entire input simultaneously and

[3] `https://doi.org/10.5281/zenodo.16127320`

are primarily used for discriminative tasks, such as classification (e.g., vulnerability prediction and clone detection). Decoder-only models (e.g., CodeGPT [23], StarCoder [31], and Codex [32]) generate output token by token and are typically employed in generative tasks, such as code generation, code completion, and summarization. Encoder-decoder models (e.g., PLBART [24] and CodeT5 [25]) encode the input into a latent representation and subsequently decode it into an output sequence, making them suitable for translation-style tasks such as code summarization and programming language translation. Among these categories, encoder only models, particularly CodeBERT and GraphCodeBERT, have been widely and effectively applied to various downstream software analytics tasks and model compression studies [6, 10, 11, 22, 33], as evidenced by prior empirical evaluations [23]. Consequently, these models serve as the primary focus of our study.

2.2. Model Compression

A significant barrier to the real-world deployment of language models of code is their substantial computational overhead, high memory demands, and environmental impact (e.g., carbon emissions) [7, 34]. To mitigate these challenges and advance sustainability, a widely adopted strategy known as *model compression* offers a solution that reduces the resource consumption of such models without significantly compromising performance. Model compression refers to the process of reducing a large, computationally intensive model into a smaller, more efficient version suitable for deployment on resource-constrained devices (e.g., consumer-grade laptops). In addition to deployment benefits, model compression improves inference speed and enhances overall resource efficiency. Algorithmically, the fundamental model compression techniques include: *pruning* [35], *quantization* [36], *knowledge distillation* [9], *neural architecture search* [37], *weight sharing & hashing* [38], and *low-rank factorization* [39]-each method uniquely contributes to model optimization. Among these, pruning, quantization, and knowledge distillation (KD) have emerged as the most prominent and widely adopted approaches for compressing language models of code in recent software engineering research. [6, 10, 12, 14, 15, 33, 40].

While pruning and quantization primarily remove redundant parameters or reduce numerical precision to achieve compactness, their ability to produce highly compact models is inherently limited, as aggressive compression often results in substantial degradation of representational capacity. In contrast, KD offers a distinct advantage by enabling the creation of a smaller student model that can closely approximate, and in some cases replicate, the predictive behavior of its larger teacher counterpart. Through this process, the student learns to mimic the teacher's functional behavior using soft targets such as probability distributions, internal embeddings, or logits generated by the teacher. This allows the student model to preserve the teacher's predictive behavior and generalization while maintaining a significantly reduced computational footprint. In the context of language models of code, KD has emerged as a promising compression technique for producing a deployable model that retains functional equivalence with its teacher, although the extent to which a student model truly mimics its teacher's behavior remains largely unexplored in software engineering.

2.3. Metamorphic Testing

Metamorphic testing (MT) is a property-based testing paradigm widely adopted for validating software systems in the absence of a test oracle or when one is difficult to define,

a challenge commonly referred to as the oracle problem [20, 41]. Rather than verifying the correctness of individual outputs in isolation, MT addresses this issue by evaluating whether the program under test exhibits expected relationships between outputs across a set of well-defined metamorphic relations (MRs). An MR specifies a necessary property that captures the expected relationship between multiple inputs and their corresponding outputs. These relations are derived from the problem domain's inherent semantics and define how the output should change (or remain consistent) when the input is systematically modified. This paradigm is beneficial for testing complex systems such as machine learning models, scientific computing software, or simulation tools, where correctness is difficult to assert for individual outputs but consistency across related cases can be meaningfully assessed [42–46].

In this study, we adapt MT to evaluate how deeply a student model produced via KD mimics its teacher model's behavior. Unlike conventional MT, we do not apply transformations to the input samples. Instead, we define MRs between the teacher and student models' outputs, ensuring that when both receive the same input, their outputs exhibit a consistent relationship. These teacher–student MRs capture multiple aspects of behavioral fidelity, including agreement in predicted labels, similarity in probability distributions, and calibration consistency. By systematically evaluating these MRs, we gain a deeper understanding of behavioral fidelity between the teacher and student models beyond accuracy-based evaluation.

2.4. Adversarial Attack

An adversarial attack refers to the deliberate process of crafting adversarial examples by introducing subtle perturbations to the original inputs. Although these perturbed instances appear almost identical to their original counterparts from a human perspective, they can cause the model to yield erroneous or inconsistent outputs. The concept of adversarial attack was first introduced by Zegedy et al. [47], who demonstrated that state-of-the-art image classifiers can be misled by carefully designed pixel-level modifications that remain imperceptible to human observers. Formally, given a classifier $f : X \to Y$ that correctly predicts the label $y_{truth} \in Y$ for an input $x \in X$, an adversarial attack generates a perturbed version x' such that $f(x') \neq f(x) = y_{truth}$.

In continuous domains such as images, producing visually indistinguishable adversarial examples is relatively straightforward. However, generating adversarial examples for discrete domains like source code is considerably more complex due to the rigid syntactic and grammatical structure of programming languages [18, 48]. As a result, adversarial examples designed to attack language models of code must meet three fundamental requirements. **First**, the perturbation must effectively mislead the model and reduce its predictive performance. **Second**, the perturbed code must remain syntactically correct, adhering to the formal rules of the programming language; for example, in Python, identifiers cannot begin with digits and may only contain letters, digits, or underscores. **Third**, the perturbed code x' must preserve the semantics of the original code x, maintaining identical functionality and behavior when executed under the same conditions. When an input instance is modified in accordance with these constraints to generate a perturbed version, the resulting instance constitutes an adversarial example; if it causes the model to produce an incorrect or inconsistent output, it is regarded as an adversarial attack.

2.5. Adversarial Robustness

Adversarial robustness describes a model's capacity to maintain its predictive performance and stability when exposed to adversarial perturbations. A model is considered

adversarially robust if it continues to generate consistent, accurate outputs despite deliberate input modifications intended to mislead it. In the context of machine learning, this concept reflects a model's resilience to adversarial attacks that exploit its sensitivity to small perturbations in the input space [47, 48].

2.6. Kullback–Leibler Divergence

The Kullback-Leibler (KL) divergence [49] is a measure of how one probability distribution Q diverges from a reference probability distribution P. For discrete distributions defined over the same support, it is mathematically expressed as:

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_{i} P(i) \log \frac{P(i)}{Q(i)} \tag{1}$$

where $P(i)$ and $Q(i)$ represent the probabilities of the i-th event under distributions P and Q, respectively. KL divergence is always non-negative and equals zero if and only if $P = Q$. It is commonly used in machine learning to quantify the difference between predicted and target distributions, particularly in tasks such as knowledge distillation. An important characteristic of KL divergence is that it is *not symmetric*, meaning that in general, $D_{\mathrm{KL}}(P \parallel Q) \neq D_{\mathrm{KL}}(Q \parallel P)$.

3. Empirical Study

This section presents the design of our empirical study, which aims to evaluate whether the student model deeply mimics the teacher's predictive behavior and internal representations during knowledge distillation by assessing its adversarial robustness. We hypothesize that if the student model deeply mimics the teacher during knowledge distillation, it should exhibit adversarial performance comparable to the teacher's. Any substantial deviation in robustness would indicate that the student model fails to deeply mimic the teacher's predictive behavior and internal representations. We first describe the selected knowledge distillation approaches for code-language models, followed by details on datasets, models, adversarial attack approaches, and the experimental setup. Finally, we outline the evaluation metrics used in the analysis and present the corresponding results in the following section.

3.1. Study Design

3.1.1. Knowledge Distillation Approaches

In recent years, several knowledge distillation (KD) approaches have been proposed for language models of code, including *Compressor* [6], *AVATAR* [10], *SODA* [15], *MORPH* [22], and *PIONEER* [33]. In this study, we select *Compressor*, *AVATAR*, and *MORPH* as our KD approaches to generate student models, as the replication package for *SODA* is not fully available[4]. Furthermore, both *MORPH* and *PIONEER* incorporate robustness as an additional objective of knowledge distillation; however, because *MORPH* optimizes a broader set of objectives than *PIONEER*, we exclude *PIONEER* from our empirical investigation.

Compressor, *AVATAR*, and *MORPH* generate deployable models from language models of code using neural architecture search guided by multi-objective optimization. *Compressor* explores various architecture-related hyperparameters, including the *number of*

[4]https://shorturl.at/Se0f7

network layers, *dimensionality of network layers*, *number of attention heads*, *dimensionality of the feed-forward layers*, and *vocabulary size*. Whereas, both *AVATAR* and *MORPH* use the same configuration space for distillation. The configuration parameters include the choice of *tokenizer* (*BPE*, *WordPiece*, *Unigram*, or *Word*), a *vocabulary size* ranging from 1,000 to 46,000, and *number of hidden layers* ranging from 1 to 12. *Hidden layer dimension* varies from 16 to 256, with *activation functions* selected from *gelu*, *relu*, *silu*, and *gelu_new*. *Hidden and attention dropout probabilities* are each set to a value between 0.2 and 0.5. *Intermediate layer size* ranges from 32 to 3,072, and *number of attention heads* ranges from 1 to 12. *Maximum sequence length* is configured between 256 and 512, while *position embedding type* can be *absolute*, *relative_key*, or *relative_key_query*.

Compressor uses the genetic algorithm to identify a simplified architecture that maximizes *giga floating-point operations* (GFLOPs) while minimizing *model size* within a fixed memory budget. *AVATAR* compresses language models of code into deployable versions by jointly optimizing *model size*, *inference latency*, *energy consumption*, and *accuracy*. It formulates the problem as a multi-objective configuration tuning task, using an SMT solver to define the search space and a tailored optimization algorithm to identify a Pareto-optimal set of configurations for the student model. Based on the *AVATAR*, *MORPH* considers *metamorphic robustness* a crucial property for multi-objective optimization in architecture selection for the student model. In addition, it distills knowledge from the teacher model not only on the soft targets but also on the metamorphic code's outputs, assuming the teacher model always produces correct outputs for the metamorphic code. Subsequently, knowledge distillation is performed by minimizing the loss between the soft targets produced by the teacher model and those generated by the student model. For example, *Compressor* employs the KL divergence loss function, defined as follows:

$$\mathcal{L} = -\frac{1}{n}\sum_{i}^{n} softmax(\frac{p_i}{T})\log\Big(softmax(\frac{q_i}{T})\Big)T^2 \tag{2}$$

In "(2)", p_i and q_i represent the teacher and student model logits, respectively, with T denoting the temperature parameter controlling the softmax distribution [9]. While the teacher logits remain fixed, the student logits are updated throughout the training process. It is essential to note that we utilize *off-the-shelf* compressed versions of CodeBERT and GraphCodeBERT, produced by *MORPH* and *AVATAR*, for clone detection and vulnerability prediction tasks in this empirical evaluation.

3.1.2. Datasets and Subjects

To minimize experimental bias in evaluating whether the student model deeply mimics the teacher's predictive behavior and internal representations during knowledge distillation, we adopt the same models and datasets used in the knowledge distillation–based approaches described in Section 3.1.1. Specifically, we select CodeBERT [16] and GraphCodeBERT [21] models and two well-studied downstream tasks: clone detection and vulnerability prediction, which serve as standard benchmarks in knowledge distillation research for software engineering [6, 10, 11, 22, 33].

Clone detection: (CD) This task aims to determine whether two given code snippets are syntactically or semantically equivalent. To streamline the fine-tuning process, we adopt the filtered BigCloneBench dataset [50] provided by Wang et al. [51]. The final dataset includes 90,102 training examples, along with 4,000 examples each for validation and testing. **Vulnerability prediction:** (VP) This task aims to determine whether a given code snippet contains a vulnerability. We conduct our experiments using the

Devign[5] dataset, introduced by Zhou et al. [52]. The dataset comprises 27,318 functions sourced from two widely used open-source C libraries, FFmpeg and Qemu. Each function is manually annotated to indicate whether it contains vulnerabilities. We use the CodeXGLUE benchmark splits [23], including training, development, and test sets with 27,318, 2,732, and 2,732 examples, respectively.

CodeBERT is a pre-trained language model that shares the same architecture and a similar pre-training strategy as RoBERTa [27]. It consists of twelve Transformer layers, each with 12-headed multi-head attention mechanisms and a hidden size of 768. CodeBERT is pre-trained using two objectives: *masked language modeling*, which predicts masked tokens, and *replaced token detection*, which identifies substituted tokens in the input. The model is trained on bimodal data comprising natural language–code pairs across six programming languages from the CodeSearchNet dataset [29]. **GraphCodeBERT** shares CodeBERT's architecture but incorporates data-flow information from source code during pre-training. Additionally, it introduces a pre-training task that predicts whether a data flow edge exists between two nodes.

3.1.3. Adversarial Attack Approaches

In this study, adversarial attacks are employed to evaluate how deeply the student model mimics the teacher's predictive behavior and internal representations during knowledge distillation. Adversarial attacks are typically categorized as black-box or white-box, depending on the attacker's level of access to the model. Following the recommendation of Du et al. [53], we adopt black-box attacks, as they rely solely on model outputs and are widely used for evaluating language models of code [17, 18]. The perturbations introduced through these attacks are deliberately subtle to maintain the syntactic and semantic integrity of the source code, as discussed in Section 2.4, while inducing mispredictions. To this end, we employ three established black-box techniques: ALERT [17], Metropolis Hastings Modifier (MHM) [18], and WIR-Random [19], each of which performs identifier substitution to assess model robustness without altering code functionality. **ALERT** [17] considers both operational and natural semantics, using greedy and genetic algorithms to select substitutions, with 30 candidates per identifier. **MHM** [18] utilizes Metropolis-Hastings sampling for iterative identifier substitution, with a maximum of 100 iterations and 30 candidates per iteration. **WIR-Random** [19] ranks identifiers based on their impact on model predictions and replaces them sequentially with randomly chosen candidates. It is important to note that only instances correctly classified before perturbation are included, ensuring that the robustness evaluation is based exclusively on valid predictions and that any observed degradation in model performance is caused solely by adversarial perturbations rather than pre-existing misclassifications [17, 18].

3.1.4. Experimental Settings

We fine-tune CodeBERT using the hyperparameter settings and training pipeline provided by the CodeXGLUE benchmark [23], while for GraphCodeBERT, we adopt the settings specified in its original paper [21]. To ensure consistency across our experiments, we adopt the data-splitting strategy used in knowledge distillation–based approaches such as *MORPH* and *AVATAR*, which divide the training data into two equal halves: one for fine-tuning and the other for knowledge distillation. We apply the same fine-tuning split to all models. Despite being trained on only half of the original dataset,

[5] `https://sites.google.com/view/devign`

all models achieve performance comparable to the state-of-the-art results reported in the CodeXGLUE benchmark [23], the GraphCodeBERT paper [21], and knowledge distillation–based compression studies such as *Compressor*, *AVATAR*, and *MORPH*.

3.1.5. Evaluation Metrics

Following the evaluation protocols of CodeXGLUE [23], Yang et al. [17], and Shi et al. [6, 10], we use **accuracy** as the primary metric to evaluate model performance on downstream tasks.

To evaluate the performance of both teacher and student models under adversarial conditions, we measure the effectiveness of the applied attack methods. A higher attack effectiveness indicates greater model vulnerability to adversarial perturbations. We employ the *Attack Success Rate* (ASR) metric [17] to measure the effectiveness of an attack method. ASR measures the proportion of adversarial samples for which the model fails to produce a correct prediction after the attack and is formally defined as follows:

$$\text{ASR} = \frac{|\{x | x \in X \wedge M(x') \neq M(x)\}|}{|X|} \tag{3}$$

Where x is an original input, x' is a modified input. A higher ASR value indicates greater attack effectiveness, reflecting the models' adversarial vulnerabilities.

The robustness of compressed models to adversarial attacks depends strongly on the quality of the adversarial examples generated. If these examples contain syntactic or semantic errors, model mispredictions may not accurately reflect a lack of robustness but rather the model's inability to handle unrealistic inputs. Moreover, excessive modifications to the original code during adversarial example generation can lead to out-of-distribution instances, thereby reducing the reliability of robustness assessments. To ensure a fair and meaningful evaluation, we conduct a qualitative analysis to assess the effectiveness of the generated adversarial examples and to verify that the applied perturbations are minimal yet sufficient to mislead the models. Because the inputs for the selected software analytics tasks are source code, we employ four established metrics from Du et al. [53]: *Identifier Change Rate* (ICR), *Token Change Rate* (TCR), *Average Edit Distance* (AED), and *Average Code Similarity* (ACS), to measure the quality of the adversarial examples.

For a set of m adversarial examples, each i-th code snippet contains k_i identifiers, among which n_i are modified to generate the adversarial example. The **ICR** is computed as follows:

$$\text{ICR} = \frac{\sum_1^m n_i}{\sum_1^m k_i} \tag{4}$$

The **TCR** represents the proportion of modified tokens relative to the total number of tokens in the original code. The **AED** quantifies the average number of character-level edits required to transform one token into another, reflecting the textual intensity of the perturbation. The **ACS** measures cosine similarity between embeddings produced by CodeBERT before and after the adversarial modifications. In general, high-quality adversarial examples are characterized by lower ICR, TCR, and AED values, along with higher ACS scores, indicating that the perturbations are subtle yet effective while preserving code correctness.

3.2. Empirical Results

This section presents the empirical results of our study designed to address the first research question, **RQ1**: *Does the student model deeply mimic the teacher's predictive behavior and internal representations during knowledge distillation?*

We address this research question by evaluating the performance of student models under adversarial attacks relative to their teacher counterparts. In addition, we examine whether traditional accuracy-based evaluation can capture discrepancies in behavioral fidelity arising from the student models' inability to deeply mimic their teacher models. Since AVATAR outperforms Compressor and MORPH considers metamorphic robustness during knowledge distillation, we compare the adversarial robustness of student models compressed by AVATAR and MORPH with that of the teacher model for this empirical analysis.

To answer this research question, we first compare the performance of the teacher and student models on the original, unperturbed test samples using accuracy as the evaluation metric. Table 1 presents the accuracy comparison between teacher and student models across the clone detection and vulnerability prediction tasks. Overall, the results indicate that the student models maintain performance levels comparable to their teacher counterparts, with accuracy differences remaining within a narrow margin (mostly below 3%). For instance, in clone detection, the student models CodeBERT and GraphCodeBERT compressed by *AVATAR* and *MORPH* achieve accuracies close to those of their teacher models, with only minor reductions observed for *AVATAR*. A similar trend is observed in vulnerability prediction, where student models exhibit minimal degradation and, in some cases, nearly identical performance to the teachers.

Table 1: Accuracy (%) comparison between teacher (uncompressed) and student (compressed) models across clone detection and vulnerability prediction tasks.

Task	**Model**	**Teacher**	**Compressor**	**AVATAR**	**MORPH**
Clone Detection	CodeBERT	96.7	95.5	93.1	95.7
	GraphCodeBERT	96.5	93.9	93.6	96.3
Vulnerability Prediction	CodeBERT	61.7	59.2	60.6	59.4
	GraphCodeBERT	60.8	60.3	59.3	59.7

Although the student models achieve task performance comparable to that of their teachers, these results provide only a surface-level view of knowledge transfer. Accuracy-based evaluation captures output correctness but may overlook discrepancies in behavioral fidelity between teacher and student models. Two models may achieve similar accuracy yet differ in representation, generalization, or response to perturbations. Hence, accuracy alone may not reveal whether the student model deeply mimics the teacher's predictive behavior and internal representations. To address this, we further examine both models under adversarial attack scenarios to gain deeper insight into their behavioral fidelity.

Before analyzing model robustness under adversarial conditions, we first evaluate the quality of the adversarial examples generated by each attack technique. Excessive or unrealistic modifications can distort the syntactic or semantic structure of the original code, leading to misleading robustness assessments. To ensure reliability, we examine the syntactic and semantic integrity of the adversarial examples produced by each attack technique, confirming that the applied perturbations are minimal yet effective. This analysis uses the CodeBERT and GraphCodeBERT models trained on the clone detection

task, with complete results for the vulnerability prediction task available in our replication package. Since all attack techniques modify only *identifiers*, the resulting code retains its original semantics. Figure 1 illustrates the quality assessment of adversarial examples generated by different attack techniques against both teacher and student models.

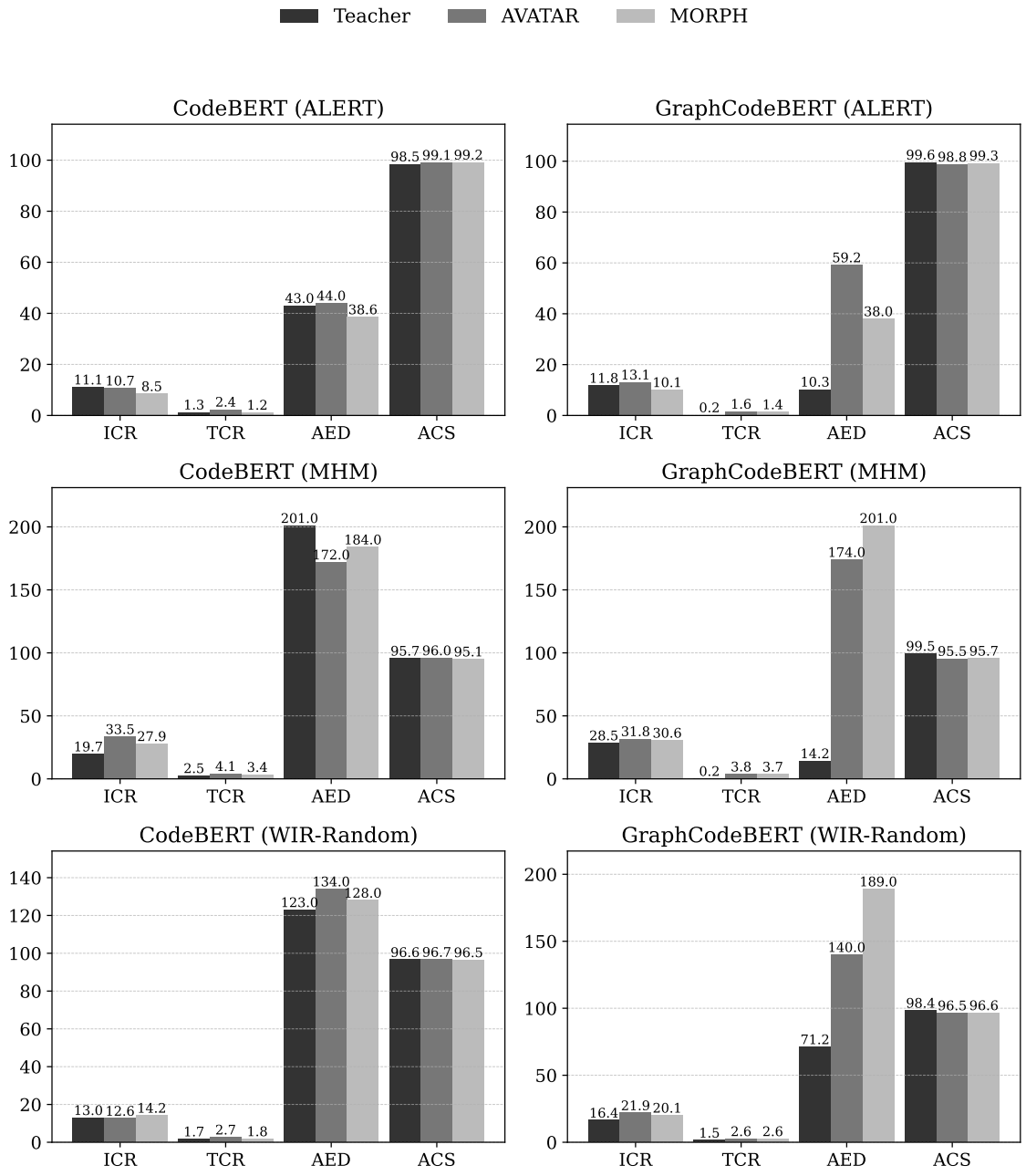


Figure 1: Quality analysis of adversarial examples across various models and evaluation metrics.

Overall, the results indicate that these attack techniques effectively preserve the syntactic integrity of the generated adversarial examples. Specifically, the examples consistently exhibit high ACS values and low TCR scores across both CodeBERT and GraphCodeBERT models, demonstrating minimal syntactical distortion. The adversarial examples also maintain strong overall quality across the AED and ICR metrics for both the teacher and student models. Although a slight degradation is observed in certain cases—for instance, the MHM attack shows comparatively higher AED values—the over-

all differences remain limited. We further perform the Friedman test [54] to determine whether the three attack techniques (ALERT, MHM, and WIR-Random) generate adversarial examples of significantly different quality for each model variant of CodeBERT and GraphCodeBERT. A p-value below 0.05 indicates that the differences in the quality of adversarial examples produced by the three attack techniques are statistically significant. However, the experimental results show no statistically significant differences among the attack methods for either CodeBERT ($p = 0.47$, $p = 0.37$, and $p = 0.37$ for Teacher, AVATAR, and MORPH, respectively) or GraphCodeBERT ($p = 0.63$, $p = 0.37$, and $p = 0.37$ for Teacher, AVATAR, and MORPH, respectively). These results indicate that all three attack techniques generate adversarial examples of comparable quality across the evaluated metrics (ICR, TCR, AED, and ACS).

Although the preceding results show that the adversarial examples essentially preserve syntactic and semantic quality across both teacher and student models, it remains important to examine whether any subtle quality differences exist between these two model categories. To investigate this, we compare the quality of adversarial examples generated by each attack technique for the teacher and student models across all the evaluation metrics. The Wilcoxon signed-rank test [55] is used to assess the statistical significance of these differences. The comparison between the Teacher and AVATAR models ($p = 0.13$) and between the Teacher and MORPH models ($p = 0.85$) indicates that the quality of adversarial examples remains statistically comparable, with no significant differences observed in either case. These findings suggest that the attack techniques generate adversarial examples of similar syntactic and semantic integrity across both teacher and student models, confirming that the subsequent robustness analysis is based on adversarial inputs of consistent quality and that any observed performance degradation can be attributed to the models' inability to maintain robustness against adversarial perturbations.

Having rigorously evaluated the quality of the generated adversarial examples, we next examine how effectively the teacher and student models withstand adversarial attacks. Figure 2 presents the Attack Success Rate (ASR) values for the teacher and student models (AVATAR and MORPH) across the ALERT, MHM, and WIR-Random attack techniques for both the clone detection and vulnerability prediction tasks. Overall, the results indicate that student models produced through knowledge distillation exhibit substantially lower robustness than their corresponding teacher models, as evidenced by consistently higher ASR values across most attack settings, with only two exceptions. In the clone detection task, the teacher model demonstrates lower robustness than the MORPH student model under the MHM and WIR-Random attacks, and than the AVATAR student model under the MHM attack. Although the robustness differences between the teacher and student models are relatively small for the GraphCodeBERT model and the vulnerability prediction task, they are considerably larger in all other experimental settings. For instance, in the clone detection task with the CodeBERT model under the ALERT attack, the AVATAR student model experiences approximately three times greater performance degradation than the teacher model.

Taken together, these findings suggest that student models distilled from their teachers, particularly those produced by AVATAR and, to a lesser extent, by MORPH, are more susceptible to adversarial attacks than their teacher counterparts. Surprisingly, although *MORPH* generates student models by incorporating metamorphic robustness during knowledge distillation, the resulting student models exhibit lower robustness than those produced by *AVATAR* across all the attack techniques for the GraphCodeBERT model in the vulnerability prediction task. The results further reveal that student mod-

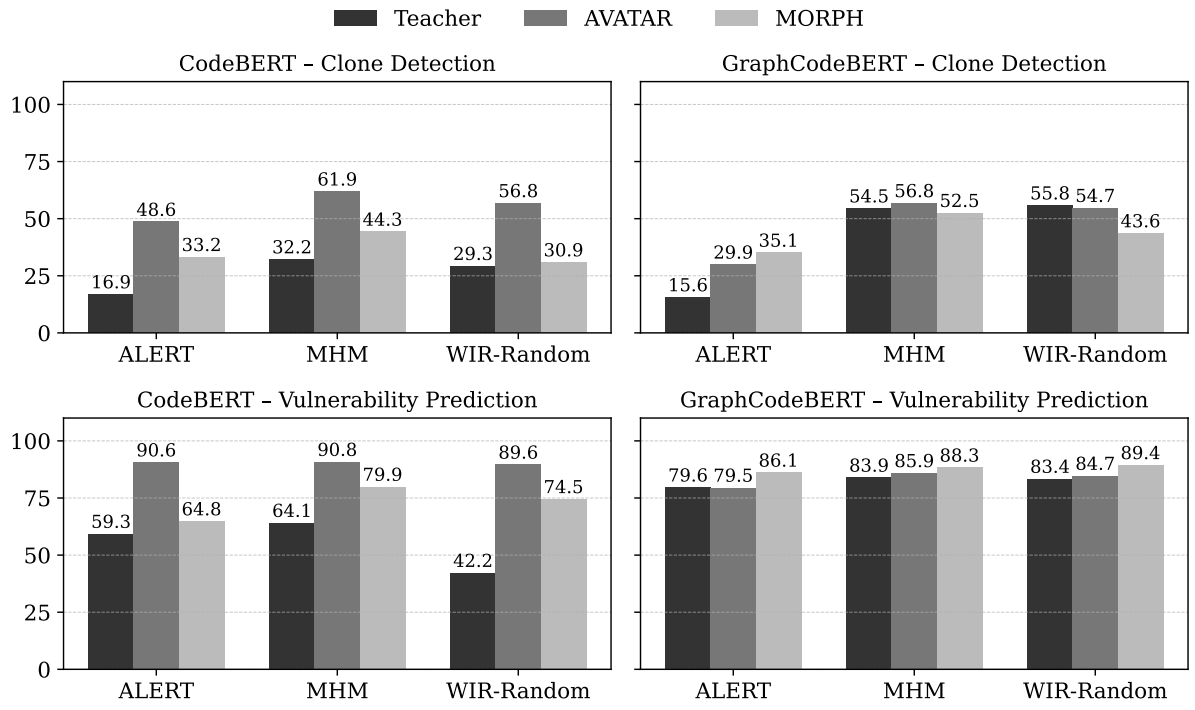


Figure 2: Attack success rate metric values across adversarial attack techniques, models, tasks, and knowledge distillation methods.

els fail to deeply mimic the teacher's predictive behavior and internal representations. Although knowledge distillation preserves task accuracy under clean inputs, it weakens the model's ability to generate stable and consistent predictions when exposed to adversarial perturbations. As a result, traditional accuracy-based evaluation cannot capture these discrepancies in behavioral fidelity, which become evident only under adversarial conditions. These observations highlight the need for more comprehensive evaluation frameworks that assess behavioral fidelity beyond accuracy.

Answer to RQ1

Since the adversarial examples across all teacher and student models exhibit comparable syntactic and semantic quality, the observed robustness gap can be attributed to the student models' inability to deeply mimic the teacher models' predictive behavior and internal representations, a limitation that traditional accuracy-based evaluation fails to reveal.

4. Approach: *MetaCompress* Framework

The empirical findings in Section 3.2 show that traditional accuracy-based evaluation is insufficient to capture behavioral discrepancies between teacher and student models, which become evident under adversarial conditions due to the student models' inability to deeply mimic their teacher counterparts' predictive behavior and internal representations. Motivated by this observation, we introduce *MetaCompress*, a metamorphic testing framework for evaluating the behavioral fidelity of compressed models by framing the task as a classical software testing problem and applying a metamorphic testing strategy. The framework identifies key behavioral invariants that should be preserved

after model compression and formalizes a set of output-based metamorphic relations that serve as oracle-free tests to assess behavioral fidelity between teacher and student models. Figure 3 presents the overall design and workflow of the proposed *MetaCompress* framework.

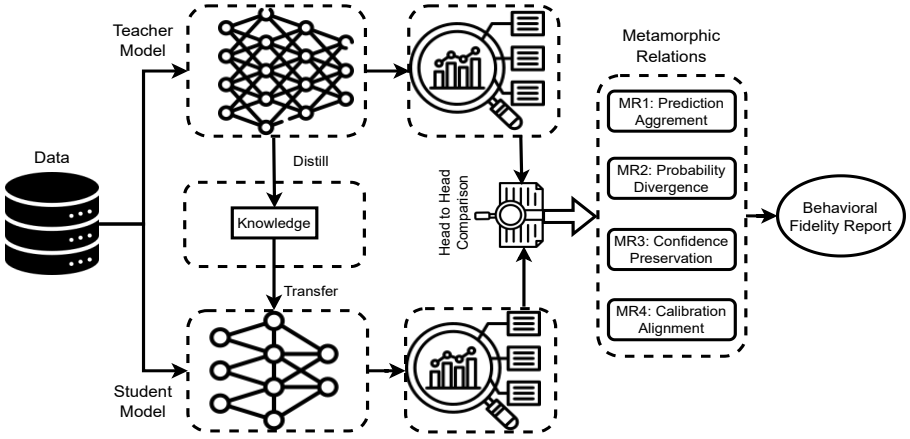


Figure 3: Workflow of the *MetaCompress* framework, comparing teacher and student model outputs under behavior-preserving metamorphic relations to assess behavioral fidelity.

4.1. Behavioral Fidelity as a Software Testing Problem

Knowledge distillation transfers knowledge from a large teacher model to a smaller student model, aiming to preserve the student's performance on the original task. However, our empirical results demonstrate that the student model does not deeply mimic the teacher's internal behavior, raising an important question: *Does the student model behave consistently with its teacher?* This concern closely parallels a classic problem in software engineering: verifying that a modified system preserves the intended behavior of its original version. In this context, we treat the student model as a transformed version of the teacher model and frame the evaluation of behavioral fidelity as a software testing problem. From this perspective, the teacher and student models represent two versions of the same system under test (SUT), and our goal is to assess whether behavioral fidelity is maintained after knowledge distillation. Formally, let $M : \mathcal{X} \to \mathcal{Y}$ denote the teacher model and $M' : \mathcal{X} \to \mathcal{Y}'$ its student variant. Given a set of test inputs $\mathcal{T} \subseteq \mathcal{X}$ and a task-specific equivalence relation $\equiv \subseteq \mathcal{Y} \times \mathcal{Y}'$, the problem of behavioral fidelity testing is to determine whether:

$$\forall x \in \mathcal{T}, \quad M(x) \equiv M'(x) \tag{5}$$

That is, whether the student model produces outputs that are behaviorally consistent with those of the teacher model across all test cases.

In traditional software engineering, techniques such as regression testing [56], mutation testing [57], and metamorphic testing [20] are commonly employed to assess the reliability of modified systems. However, evaluating language models of code presents unique challenges compared to traditional software systems. For example, traditional software typically produces deterministic outputs, while language models of code yield probabilistic outputs, and knowledge distillation may alter output distributions even when top-1 predictions remain unchanged. Additionally, we aim to assess the behavioral fidelity of a student model compared to its teacher counterpart. Although ground-truth labels are

available for tasks such as clone detection or vulnerability prediction, our objective is not to directly compare the outputs of the two models against the ground-truth. Instead, we compare their outputs after model execution, treating the teacher model as the behavioral reference. Notably, the teacher model's behavior is not known until it is executed on test data. We refer to this scenario as an instance of the *oracle problem*, where the absence of a definitive oracle hinders straightforward correctness verification.

As a result, conventional software testing techniques such as regression testing, which depend on exact output matches, are often too rigid—minor differences in probabilistic outputs or generated sequences do not necessarily indicate functional errors. Mutation testing is also ill-suited, as neither model is inherently faulty. In contrast, metamorphic testing provides a principled approach to overcoming the oracle problem by defining expected relations (as detailed in Section 4.2) between the outputs of the teacher and student models, without requiring explicit ground-truth. These relations act as relational oracles, enabling us to verify whether the teacher and student models produce consistent outputs to the same inputs. Finally, metamorphic testing has been widely adopted for evaluating machine learning–based software systems and large language models [42, 58–62], demonstrating its versatility and effectiveness in oracle-free evaluation scenarios. Building on this foundation, *MetaCompress* adopts a metamorphic testing–based approach to evaluate behavioral fidelity of student models, framing it as an oracle-free testing problem.

4.2. Designing Output-Based Metamorphic Relations

MetaCompress is designed to evaluate discrepancies in behavioral fidelity between teacher and student models. Without a head-to-head comparison of teacher and student models on identical inputs, it is difficult to determine whether the student model maintains the teacher model's behavioral fidelity. For example, a manual inspection of a student GraphCodeBERT model generated using *Compressor* for the vulnerability prediction task illustrates this limitation. Figure 3(a) shows a case in which both models predict the same class, yet their output probability distributions diverge substantially, with a Kullback–Leibler divergence of 0.6177. In contrast, as shown in Figure 3(b), the teacher and student models yield different class predictions for the same input, revealing behavioral differences that accuracy alone fails to capture. To systematically assess such differences, *MetaCompress* defines a set of output-based metamorphic relations that encode expected behavioral invariants, such as class agreement and similarity in probability distributions, between teacher and student models. Unlike traditional metamorphic testing, which typically applies transformations to inputs, these relations operate directly on model outputs, enabling a structured and quantitative evaluation of behavioral fidelity. The following subsections describe the design rationale, empirical motivation, and evaluation metrics for each metamorphic relation incorporated into the *MetaCompress* framework.

4.2.1. MR1 - Prediction Agreement

Description. Given a set of test inputs $X = x_1, x_2, ..., x_n$, a teacher model M_T and a student model M_S, for each input $x_i \epsilon X$, the prediction of M_S should match of M_T. We define **MR1** as follows:

$$M_S(x_i) = M_T(x_i) \tag{6}$$

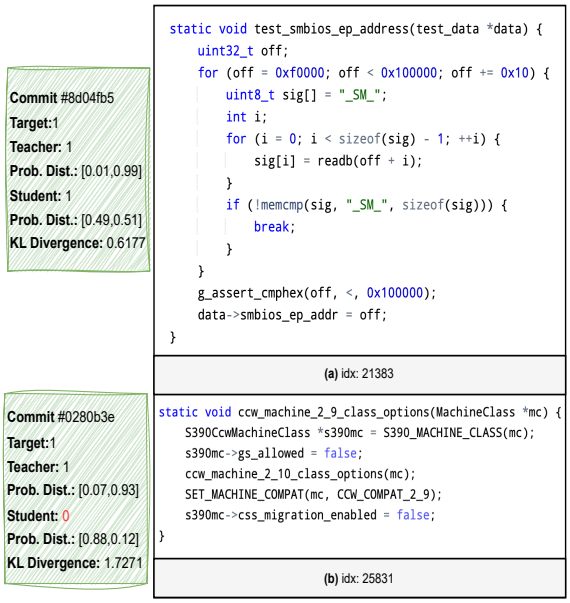


Figure 4: Behavioral fidelity discrepancies between the outputs of teacher and student models.

Rationale. This MR captures the most intuitive form of behavioral fidelity: *top-1* label consistency. Our empirical observations (as shown in Figure 3(b)) suggest that prediction disagreement may be associated with divergence in model reasoning. Therefore, **MR1** serves as a meaningful indicator of behavioral fidelity.

Evaluation metric. To measure this MR, we consider the *Label Loyalty* metric that quantifies the degree to which this MR holds across all the test instances. It computes the percentage of instances for which the student model's prediction matches the teacher model's prediction, analogous to accuracy:

$$\text{Loyalty}_{\text{lab}} = \frac{1}{n}\sum_{i=1}^{n} 1[M_S(x_i) = M_T(x_i)] \tag{7}$$

Thus, $\text{Loyalty}_{\text{lab}}$ quantitatively measures the success rate of this MR. A higher metric value indicates greater behavioral fidelity preservation in the student model.

4.2.2. MR2 - Probability Distribution Similarity

Description. This MR assesses the similarity of output probability distributions between the two models by evaluating whether, for a given input x, the predicted distribution of M_S closely aligns with that of M_T. Given a set of test inputs $X = x_1, x_2, ..., x_n$, let $P_T(y \mid x)$ and $P_S(y \mid x)$ represent the probability distributions over the output space $\mathcal{Y}$ produced by M_T and M_S, respectively. We define **MR2** as follows:

$$D\left(P_T(\cdot \mid x), P_S(\cdot \mid x)\right) \leq \delta, \quad \forall x \in X \tag{8}$$

Where $D(\cdot, \cdot)$ is a divergence metric (e.g., Kullback–Leibler divergence), and $\delta \geq 0$ is a predefined tolerance threshold.

Rationale. Although the teacher and student models may produce the same predicted label, their underlying probability distributions can differ substantially. Our empirical observations (Figure 3(a)) reveal such differences, as measured by the Kullback–Leibler

divergence metric (e.g., 0.6177), suggesting that prediction agreement (**MR1**) alone is insufficient to ensure behavioral fidelity.

Evaluation metric. To quantify the difference between the two probability distributions, we use the Kullback–Leibler divergence, as defined in "(1)". Given a tolerance $\delta = 0.5$ (Inspired by sigmoid-based methods for binary classification [63]), we then count the test instances for which MR2 holds:

$$\text{Loyalty}_{\text{prob}} = \frac{1}{|X|} \sum_{x \in X} \mathbf{1}\big[D_{\text{KL}}\big(P_T(\cdot \mid x), P_S(\cdot \mid x)\big) \leq \delta\big] \tag{9}$$

A larger $\text{Loyalty}_{\text{prob}}$ indicates that the student model retains the teacher model's probability-distribution behavior on a greater portion of the test set.

4.2.3. MR3 - High Confidence Preservation

Description. Given a set of test inputs $X = x_1, x_2, ..., x_n$, M_T, and M_S, consider the subset: $\mathcal{X}_{\text{conf}} = \{\, x_i \in X \mid \max M_T(x_i) \geq \tau \,\}$,

where τ is a high-confidence threshold (e.g., $\tau = 0.9$). For every $x_i \in \mathcal{X}_{\text{conf}}$, the student model should maintain

$$\arg\max M_S(x_i) \;=\; \arg\max M_T(x_i) \quad \wedge \quad \max M_S(x_i) \;\geq\; \tau \tag{10}$$

In other words, M_S must predict the same top-1 class as M_T *and* do so with confidence at least τ.

Rationale. A Top-1 agreement alone may mask cases where M_S is hesitant while M_T is decisive. Our manual investigation (Figure 3(a)) revealed that large confidence gaps (e.g., $|0.99 - 0.51| = 0.48$) often coincide with divergent reasoning, even when predictions align. By enforcing both label and confidence alignment on high-certainty inputs, **MR3** probes a deeper layer of behavioral fidelity.

Evaluation metric. We introduce the *High Confidence Agreement Rate* (HCAR) metric to quantify how often this relation holds:

$$\text{HCAR} = \frac{1}{|\mathcal{X}_{\text{conf}}|} \sum_{x \in \mathcal{X}_{\text{conf}}} \mathbb{I}\left[\begin{array}{l} \arg\max T(x) = \arg\max S(x) \\ \wedge \;\; \max S(x) \geq \tau \end{array}\right] \tag{11}$$

Here $\mathbb{I}[\cdot]$ denotes the indicator function, which returns 1 if the condition is satisfied and 0 otherwise. A higher HCAR indicates that the student model accurately preserves the teacher's confident and decisive behavior, whereas a lower value suggests reduced fidelity after knowledge distillation.

4.2.4. MR4 - Calibration Alignment

Description. Given a set of test inputs $X = x_1, x_2, ..., x_n$, M_T, and M_S, we partition the confidence interval $[0, 1]$ into B equal-width bins. For each bin $i \in \{1, \ldots, B\}$, let

$$\text{acc}_i^{\text{orig}} = \frac{1}{|X_i|} \sum_{x \in X_i} 1\big[\arg\max M_T(x) = y(x)\big] \tag{12}$$

$$\text{acc}_i^{\text{comp}} = \frac{1}{|X_i|} \sum_{x \in X_i} 1\big[\arg\max M_S(x) = y(x)\big] \tag{13}$$

where X_i is the subset of inputs whose M_T confidence $\max M_T(x)$ falls into bin i and $y(x)$ is the ground-truth label. The student model is expected to preserve calibration if, for every bin i, its empirical accuracy closely matches that of the teacher.

Rationale. Top-1 agreement alone does not indicate whether the student model is *well-calibrated*. Poor calibration, where predicted probabilities do not reflect true likelihoods, can lead to over- or under-confidence, which adversely affects downstream decision-making. Prior studies [64, 65] on vision and language models demonstrate that compression techniques such as quantization can significantly distort model calibration, even when accuracy remains unaffected. These findings motivate the design of this MR to assess behavioral fidelity through the lens of calibration alignment.

Evaluation metric. We propose the *Expected Calibration Alignment* (ECA) metric, defined as the mean absolute difference in bin-wise accuracy to assess this behavioral fidelity:

$$\text{ECA} \;=\; \frac{1}{B}\sum_{i=1}^{B}\Big|\,\text{acc}_i^{\text{orig}} - \text{acc}_i^{\text{comp}}\Big| \tag{14}$$

A lower ECA score ($\text{ECA} \in [0,1]$) indicates that the student model's calibration behavior closely aligns with that of the teacher model. Conversely, a higher ECA score reflects greater divergence between the two models' calibration profiles.

Violation rate computation: To compute the violation rate of a metamorphic relation MR_k, we use the complement of its hold rate. Let $\text{HoldRate}_{\text{MR}_k} \in [0,1]$ denote the proportion of test inputs for which MR_k holds. Then, the violation rate is defined as:

$$\text{ViolationRate}_{\text{MR}_k} = 1 - \text{HoldRate}_{\text{MR}_k} \tag{15}$$

This formulation enables us to quantify the extent to which a student model deviates from the behavioral invariants defined by each metamorphic relation. Higher violation rates indicate greater behavioral drift from the teacher model.

Expected Behaviour: A student model is considered behavior-preserving if, for every test input, the student model: (i) produces the exact top-1 prediction as the teacher; (ii) retains a probability distribution that closely matches the teacher's in terms of uncertainty and class ranking; (iii) replicates the teacher's high-confidence decisions on unambiguous inputs (e.g., $\max M_T(x)! \geq! \tau$); and (iv) maintains aligned calibration curves such that the bin-wise accuracy difference $|\text{acc}_i^{\text{orig}} - \text{acc}_i^{\text{comp}}|$ remains consistently small.

4.3. MetaCompress Evaluation

To evaluate the effectiveness of the proposed *MetaCompress* framework, we perform an extensive empirical study using the models, datasets, and knowledge distillation techniques described in Section 3.1. This evaluation aims to determine whether *MetaCompress* can uncover behavioral discrepancies between teacher and student models that traditional accuracy-based metrics fail to capture.

4.3.1. Implementations

MetaCompress is implemented in Python on top of the *Compressor*, *AVATAR*, and *MORPH*. All experiments are conducted on an Ubuntu 22.04 machine equipped with an Intel(R) Xeon(R) W-2265 CPU @ 3.50GHz, 256 GB RAM, and one NVIDIA RTX A6000 48 GB GPU.

4.3.2. Evaluation Results

This section presents the empirical evaluation results of the *MetaCompress* framework to address the second research question, **RQ2**: *To what extent does MetaCompress uncover behavioral discrepancies that may not be captured by the traditional accuracy metric?*

We address this research question by assessing the effectiveness of the *MetaCompress* framework by measuring the degree of violation across the four designed metamorphic relations. Each violation indicates a behavioral inconsistency between the teacher and student models when exposed to identical inputs, thus quantifying the loss of behavioral fidelity after knowledge distillation. We conduct this analysis across two language models of code (CodeBERT and GraphCodeBERT), two software analytics tasks (clone detection and vulnerability prediction), and three knowledge distillation approaches (*Compressor*, *AVATAR*, and *MORPH*).

MR1 violation: The results for **MR1** violation shown in Figure 5 reveal that student models exhibit low violation rates for the clone detection task but considerably higher rates for vulnerability prediction. For clone detection, all three distillation techniques maintain strong label agreement with the teacher, with a maximum violation of 7%, where *MORPH* achieves the lowest (3%) and *AVATAR* the highest (7%) inconsistency. In contrast, violation rates rise sharply for vulnerability prediction, ranging from 29% to 36%, indicating notable behavioral divergence between teacher and student models. Among the methods, *AVATAR* performs slightly better for *CodeBERT*, while *MORPH* and *Compressor* show comparable fidelity across both models. As shown in Table 1, although the student models achieve comparable performance on the vulnerability prediction task (with a maximum accuracy difference of approximately 3%), the *MetaCompress* framework reveals up to 36% discrepancies in prediction alignment, demonstrating its ability to capture behavioral differences that traditional accuracy-based evaluation overlooks.

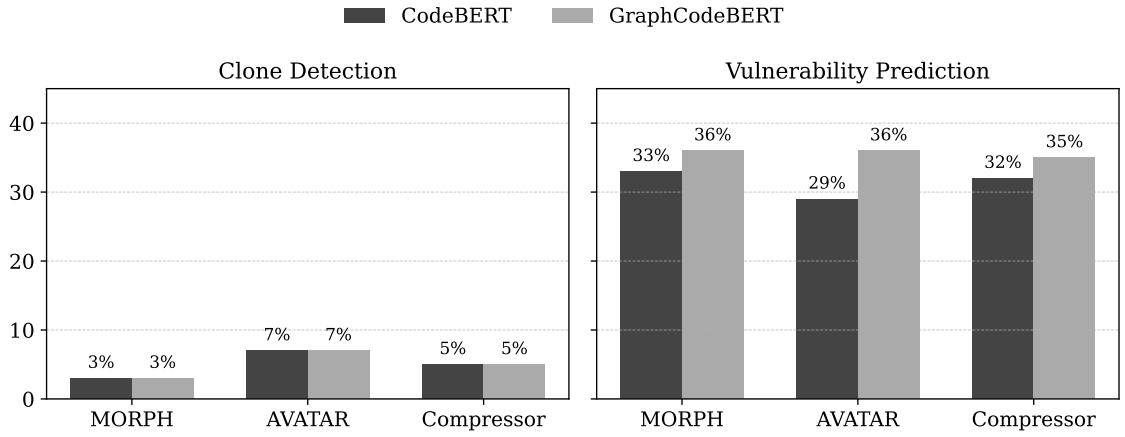


Figure 5: MR1 violation rates for different tasks, models, and knowledge distillation techniques.

MR2 violation: The results for **MR2** violations shown in Figure 6 reveal that the degree of probabilistic divergence between teacher and student models varies across tasks and distillation techniques. For the clone detection task, the violation rates remain relatively low, ranging from 3% to 13%, suggesting that the student models generally approximate the teacher's output distributions. In contrast, the vulnerability prediction task shows substantially greater violations, reaching up to 31%, reflecting a pronounced

mismatch in the probability distributions of the teacher and student models. These findings suggest that, although the student models can match the teacher's predictions in terms of label accuracy, their underlying probability distributions differ significantly, underscoring *MetaCompress* 's ability to expose fine-grained probabilistic discrepancies beyond traditional accuracy-based evaluation.

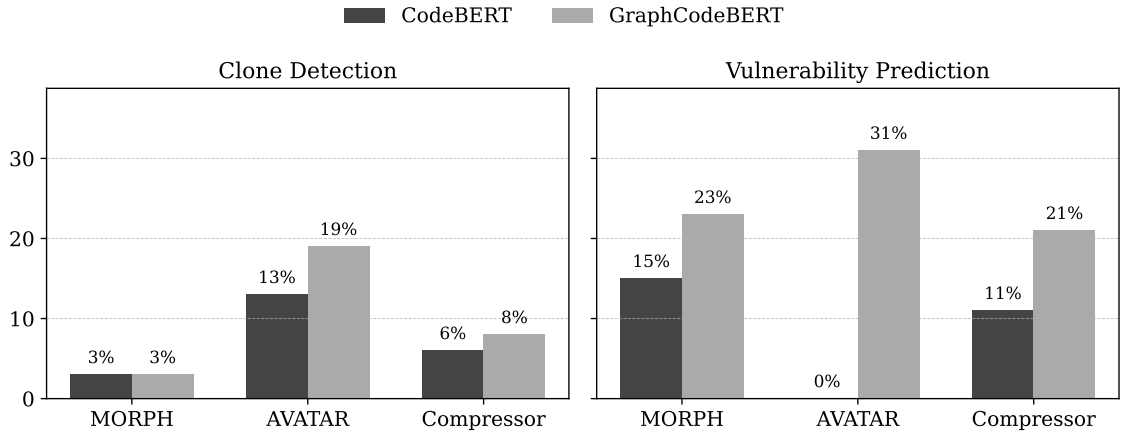
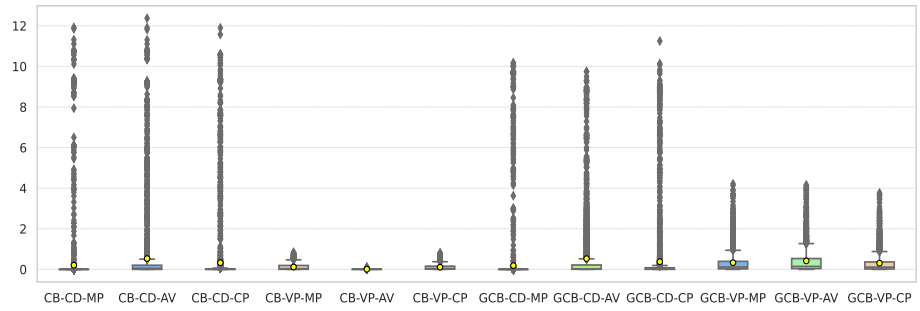


Figure 6: MR2 violation rates for different tasks, models, and knowledge distillation techniques.

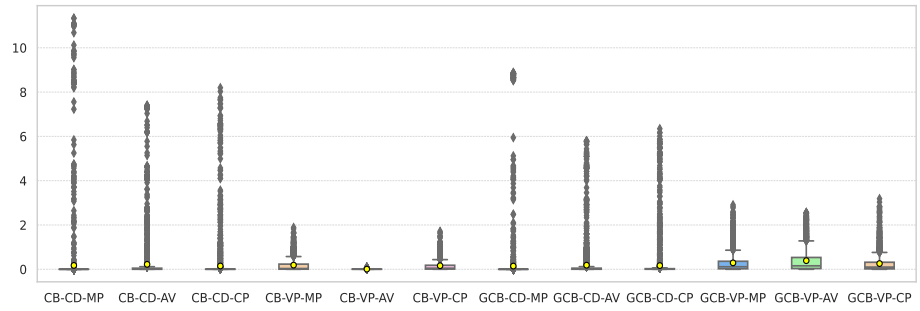
To gain a finer insight into behavioral fidelity, we visualize the distributions of MR2 violations using box plots, enabling a head-to-head comparison between teacher and student models in terms of output probability deviation across individual test instances. Given the asymmetry of KL divergence, we report both $D_{\mathrm{KL}}(P \parallel Q)$ and $D_{\mathrm{KL}}(Q \parallel P)$, where P and Q are the output probability distributions of the teacher and student models. Figure 7 presents the per-sample distributions of these values for KD approaches: *Compressor*, *AVATAR*, and *MORPH*. We observe that for a large proportion of samples, the KL divergence remains low, indicating close alignment between the teacher and student models. However, a notable number of outliers with high divergence suggest that in some cases, the student model assigns significantly different probabilities to the same output classes. These distributions underscore the utility of MR2 for capturing probabilistic distortions that may not be reflected in the *accuracy* metric. Together, the distributions of $D_{\mathrm{KL}}(P \parallel Q)$ and $D_{\mathrm{KL}}(Q \parallel P)$ provide a comprehensive understanding of discrepancies in behavioral fidelity at the granularity of individual predictions. These findings demonstrate that *MetaCompress* effectively exposes subtle differences in probabilistic reasoning between teacher and student models that remain hidden under accuracy-based evaluation.

MR3 violation: While MR1 and MR2 expose surface-level inconsistencies, such as label disagreement and output distribution shifts, MR3 uncovers deeper deviations in behavioral fidelity. Table 2 illustrates the violations of MR3 observed in student models, as identified by the *MetaCompress*. Across all tasks and student models, we observe substantial violations of MR3. As shown in Table 2, student GraphCodeBERT models often exhibit significantly lower confidence than their teacher counterparts for the vulnerability prediction task, despite having similar accuracy. For example, the GraphCodeBERT model obtained from *AVATAR* violates up to **62%** of MR3, indicating a significant dis-

[7]CB: CodeBERT, GCB: GraphCodeBERT, VP: Vulnerability Prediction, CD: Clone Detection, AV: *AVATAR*, CP: *Compressor*, MP: *MORPH*, CB-CD-AV: CodeBERT compressed via *AVATAR* for the Clone Detection task.



(a) $D_{\mathrm{KL}}(P \parallel Q)$



(b) $D_{\mathrm{KL}}(Q \parallel P)$

Figure 7: Boxplots of KL divergence illustrating output distribution shifts between teacher and student models across different tasks and knowledge distillation techniques [7].

crepancy in high-confidence predictions and overall behavioral fidelity between the teacher and student models that accuracy-based evaluation fails to capture. We further examine the sensitivity of MR3 to different threshold values of τ and find that variations in τ had a negligible impact on overall violation rates, confirming the metric's stability.

Table 2: Behavioral fidelity discrepancies in student models revealed through MR3 violations.

Task	Model	Compressor			AVATAR			MORPH		
		$\tau = 0.8$	$\tau = 0.85$	$\tau = 0.9$	$\tau = 0.8$	$\tau = 0.85$	$\tau = 0.9$	$\tau = 0.8$	$\tau = 0.85$	$\tau = 0.9$
CD	**CB**	0.03	0.03	0.03	0.06	0.05	0.04	0.03	0.03	0.03
	GCB	0.04	0.04	0.04	0.03	0.03	0.03	0.03	0.03	0.03
VP	**CB**	–	–	–	–	–	–	–	–	–
	GCB	**0.44**	**0.45**	**0.43**	**0.6**	**0.61**	**0.62**	**0.5**	**0.51**	**0.49**

Cells highlighted with a dashed line indicate cases where the teacher models do not produce any output with a high confidence score for the given task.

MR4 violation: Table 3 presents the calibration deviation between teacher and student models across three knowledge distillation techniques for varying bin sizes ($B = 10, 15, 20$). The results reveal that the calibration alignment varies notably across tasks and distillation strategies. For the clone detection task, all student models maintain near-perfect calibration consistency with their teachers, with deviation values below 0.04, indicating that the students effectively preserve the teachers' confidence distributions. In contrast, the vulnerability prediction task exhibits substantially higher deviations marked by dark gray cells, up to 0.14 for CodeBERT and 0.11 for GraphCodeBERT, demonstrating that the student models' probabilistic calibration diverges considerably from that of the teachers. These results highlight that although student models may achieve comparable accuracy, their confidence calibration can differ sharply, underscoring *MetaCompress* 's ability to capture nuanced behavioral discrepancies beyond what traditional evaluation

metrics can reveal. We further examine the sensitivity of **MR4** to different bin sizes and observe that variations in B had a negligible effect on overall violation rates, confirming the metric's stability.

Table 3: Behavioral fidelity discrepancies in student models revealed through MR4 violations.

Task	Model	Compressor			AVATAR			MORPH		
		B = 10	B = 15	B = 20	B = 10	B = 15	B = 20	B = 10	B = 15	B = 20
CD	CB	0.017	0.017	0.017	0.035	0.035	0.035	0.002	0.002	0.002
	GCB	0.026	0.026	0.026	0.063	0.063	0.063	0.003	0.003	0.003
VP	CB	**0.136**	**0.142**	**0.143**	0.016	0.016	0.017	**0.133**	**0.138**	**0.139**
	GCB	0.092	0.092	0.092	**0.112**	**0.112**	**0.112**	0.094	0.096	0.098

Answer to RQ2

MetaCompress provides a comprehensive evaluation of behavioral fidelity in language models for code, revealing substantial discrepancies that remain hidden under accuracy-based evaluation. Even when the accuracy difference between teacher and student models is within **3%**, the framework identifies up to **62%** behavioral fidelity violations across the four metamorphic relations. By analyzing model behavior from label agreement (**MR1**) to probabilistic divergence (**MR2**), confidence consistency (**MR3**), and calibration alignment (**MR4**), *MetaCompress* captures discrepancies across multiple behavioral dimensions. These findings confirm that *MetaCompress* effectively uncovers behavioral differences between teacher and student models that conventional accuracy metrics fail to reveal.

5. Discussion

Ba et al. [66] stated that "*If the student model learns to mimic the teacher model perfectly, it makes exactly the same predictions and mistakes as the teacher model.*" However, achieving such mimicry is extremely challenging when there exists a substantial gap in network capacity [67]. For instance, the knowledge distillation technique *Compressor* reduces the size of the *CodeBERT* model from 476 MB to only 3 MB. Consequently, although the student models produced by *Compressor*, *AVATAR*, and *MORPH* achieve accuracy comparable to the teacher, their performance deteriorates considerably under adversarial attacks. Even though *MORPH* attempts to distill knowledge more robustly by incorporating metamorphic attacks through metamorphic code transformations, the resulting student models still exhibit notable adversarial vulnerabilities. These observations indicate that the knowledge distillation strategies employed by *Compressor*, *AVATAR*, and *MORPH* do not enable the student models to deeply mimic the teacher's behavior, leaving a substantial *knowledge gap* in behavioral fidelity between the teacher and the student models.

One might argue that a student model performs comparably to its teacher after knowledge distillation. Then, *why should we be concerned about the knowledge gap in behavioral fidelity*? We address this question through two key arguments.

1. If the knowledge gap can be reduced by enabling the student model to more deeply mimic the teacher model, then the student should exhibit comparable robustness to its teacher under adversarial conditions. This would make the student model suitable for deployment in real-world software systems where safety and reliability are critical.

2. Wang et al. [68] empirically demonstrated that the same student model distilled from a teacher performs significantly better than when trained using the standard fine-tuning approach on the same task. Since the teacher model typically possesses much greater network capacity, it can learn more complex decision boundaries and have greater robustness against adversarial attacks. Minimizing the *knowledge gap*, therefore, enables the distilled student model to better approximate these boundaries during distillation, potentially enhancing its resilience to adversarial attacks.

The empirical findings from our *MetaCompress* framework substantiate these arguments. Although the accuracy difference between teacher and student models remains within 3%, *MetaCompress* reveals violations of up to 62% across the four metamorphic relations, indicating substantial discrepancies in behavioral fidelity. For instance, the high MR1 and MR2 violation rates demonstrate that the student models often diverge from the teacher in both label agreement and probability distribution, even when their predictions appear similar at the aggregate level. Likewise, MR3 and MR4 violations expose inconsistencies in confidence and calibration alignment, underscoring that accuracy alone cannot fully capture behavioral divergence. These results confirm that reducing the *knowledge gap* in behavioral fidelity is essential for developing distilled models that preserve the robustness and decision consistency of their teacher counterparts. These insights open promising avenues for advancing knowledge distillation methods that achieve greater alignment between student and teacher models in terms of predictive behavior, internal representations, generalizability, and robustness.

6. Threats to Validity

This section provides a brief overview of the construct and the internal and external threats associated with our study.

6.1. Internal Validity

A potential threat to internal validity may arise if differences in performance between the student and teacher models are influenced by inconsistent training procedures, fine-tuning configurations, or data handling, rather than the knowledge distillation approaches themselves. To mitigate this threat, we applied uniform training protocols and hyperparameter settings across all models, both student and teacher, following the configurations reported in prior knowledge distillation studies [6, 10, 22]. Additionally, we used identical data splits for fine-tuning and evaluation to ensure consistency and fairness in comparison. Maintaining such a consistent experimental setup enhances the internal validity of our findings by ensuring that the observed effects are due to the student model's inability to deeply mimic the teacher model's behavior, rather than differences in hyperparameters or data partitions. Additionally, the experimental results may be influenced by potential errors in implementing the training pipelines for both student and teacher models. To mitigate this threat, we adopted the standardized training procedures provided by the *CodeXGLUE* benchmark for teacher model fine-tuning and evaluation. For knowledge distillation, we utilized publicly available, off-the-shelf student models obtained from *Compressor*[8], *AVATAR*[9], and *MORPH*[10].

[8]https://github.com/soarsmu/Compressor
[9]https://github.com/soarsmu/Avatar
[10]https://github.com/apanichella/morph-distillation/tree/main

6.2. Construct Validity

A potential threat to construct validity lies in the assumption that the four metamorphic relations accurately capture behavioral fidelity. To mitigate this, the relations were not defined arbitrarily; instead, they were grounded in empirical observations of how student models diverge from their teacher counterparts on identical inputs (Figure 4). These observed patterns informed the design of each relation, ensuring that they capture behaviorally meaningful differences between student and teacher models. A potential threat to construct validity arises from framing behavioral fidelity evaluation as an oracle problem, which assumes the absence of ground-truth labels despite the availability of test inputs. To mitigate this, we argue that the outputs of the student and the teacher models are not known until execution, much like the behavior of a software system is observed only at runtime. This reinforces the validity of framing behavioral fidelity evaluation as an oracle problem. Another potential threat to construct validity arises from the inherent randomness in model behavior, which can lead to misleading conclusions about behavioral differences. To address this issue, we fixed the random seeds and conducted head-to-head comparisons between each student model and its teacher counterpart. The comparisons used identical inputs across all four metamorphic relations to ensure that any observed differences were due to the student model's inability to deeply mimic the teacher model's behavior rather than random variation.

6.3. External Validity

A potential external threat is the generalizability of *MetaCompress* beyond program understanding to tasks such as code summarization. This study focuses exclusively on knowledge distillation-based approaches, including *Compressor*, *AVATAR*, and *MORPH*, which are designed for code-language models and operate on the teacher model's logits. Since these methods compute the loss between teacher and student logits for knowledge distillation, they are not directly applicable to generation tasks, where it is uncommon to define a loss function over generated sequences that typically differ in length between the teacher and the student models. This limitation represents an inherent constraint of the knowledge distillation approaches considered in this study. Nevertheless, because *MetaCompress* evaluates behavioral fidelity based on model outputs rather than internal architectures, it remains applicable to generation tasks once knowledge distillation techniques are developed for those scenarios. The choice of models, tasks, and knowledge distillation approaches may limit the generalizability of our results. To mitigate this, we employed widely adopted models (CodeBERT, GraphCodeBERT), standard tasks (clone detection, vulnerability prediction), and representative knowledge distillation techniques (e.g., *Compressor*, *AVATAR*, and *MORPH*). Future work may extend our analysis to additional models and tasks. To facilitate such efforts, we have thoroughly documented our framework design and evaluation setup and publicly released all code and scripts used in our experiments.

7. Related Work

A substantial body of research on model compression has emerged in the fields of computer vision and natural language processing (NLP), aiming to develop large language models that are energy efficient, environmentally sustainable, and optimized for memory usage [28, 34, 69–84]. To this end, several techniques, including *pruning* [35], *quantization* [36], and *knowledge distillation* (KD) [9], have been widely explored. More

recently, these compression techniques have been extended to software analytics to improve the efficiency of language models of code. Wei et al. [12] evaluated quantized models on code generation tasks, examining accuracy, resource consumption, and environmental impact, and found that quantization can substantially improve model efficiency with minimal degradation in accuracy or robustness when applied under suitable conditions. Saad et al. [13] introduced *ALPINE*, a pruning method that is independent of programming language and effectively reduces the computational cost of code-language models. Aloisio et al. [14] conducted an empirical study on how different compression strategies, such as pruning, quantization, and knowledge distillation, affect the performance and resource efficiency of the CodeBERT model across three software analytics tasks: code summarization, vulnerability detection, and code search. Afrin et al. [40] empirically examined the effects of quantizing large code models beyond functional correctness, focusing on qualitative aspects and static properties of the generated code. While pruning and quantization improve efficiency, their compression capacity is limited, as excessive compression often harms model performance. Hence, this study focuses on knowledge distillation, which enables extremely compact models without compromising significant predictive performance.

Shi et al. [6] proposed *Compressor*, a KD-based method evaluated on CodeBERT and GraphCodeBERT for two code understanding tasks: vulnerability prediction and clone detection. Experimental results show that *Compressor* significantly reduces inference time and model size, down to 3 MB, while preserving model effectiveness. In a follow-up study, Shi et al. [10] introduced *AVATAR*, another KD-based approach designed to mitigate energy consumption and carbon emissions, while also maintaining model effectiveness and reducing inference latency. Building on AVATAR, MORPH [22] was proposed as a KD method that combines metamorphic testing with many-objective optimization to enhance model robustness on metamorphic code, as well as accuracy, size, and efficiency. Experiments on CodeBERT and GraphCodeBERT showed that MORPH achieved up to 73% greater robustness and 38% greater efficiency than AVATAR, without sacrificing accuracy or increasing model size. However, in *MORPH*, the metamorphic attack using a metamorphic sample technically differs from the adversarial attack using an adversarial example. Metamorphic attacks are performed only once on the metamorphic sample, whereas adversarial attacks are performed by repeatedly querying the model, modifying the original sample to assess the model's actual robustness. Our empirical evaluation also reveals that the student models obtained from *MORPH* experience greater performance degradation under adversarial attacks than under metamorphic attacks, compared to the student models obtained from *AVATAR*.

Recently, Liu et al. [33] proposed a KD-based approach, PIONEER, that enhances the robustness of compressed code models while preserving accuracy. Using a solver-constrained genetic algorithm and robustness evaluation with ALERT-generated examples, it achieved an 87.54% robustness gain with only a 1.67% drop in accuracy across CodeBERT, GraphCodeBERT, and CodeT5. Recently, Wang et al. [68] empirically evaluated the effectiveness of logit-based and feature-based KD approaches for code understanding tasks. Their study showed that KD consistently outperforms fine-tuning on code understanding tasks, with feature-based methods and code-specific teachers yielding the best results.

Despite these advances in applying knowledge distillation to software analytics, a significant research gap remains in understanding how deeply the student model mimics the teacher's predictive behavior and internal representations during distillation. Existing

approaches primarily rely on standard performance metrics (e.g., accuracy, precision, recall, F1-score, ROUGE, and BLEU-4) to evaluate the performance between student and teacher models. Although these metrics provide insight into task-level performance, they fail to capture how well the student model mimics the teacher's internal behavior or decision-making patterns. To date, no study has systematically examined whether a student model deeply mimics its teacher's behavioral characteristics. This limitation underscores a critical gap in current evaluation practices—one that this study addresses by introducing a structured metamorphic testing framework to assess behavioral fidelity between teacher and student models in knowledge distillation.

8. Conclusion

This study investigated the extent to which smaller student models mimic the predictive behavior and internal representations of their larger teacher counterparts in knowledge distillation. Our findings showed that, although student models achieved performance comparable to that of their teachers, their performance deteriorated significantly under adversarial attacks. This degradation underscores their limited ability to deeply mimic the teacher models' decision-making behavior and internal representations, a limitation that traditional accuracy-based evaluations fail to capture.

To address this limitation, we introduce *MetaCompress*, a metamorphic testing framework that extends behavioral fidelity assessment beyond *accuracy* by framing it as a classical software testing problem and applying metamorphic testing principles. Extensive experiments across diverse language models of code, software engineering tasks, and knowledge distillation techniques show that *MetaCompress* effectively uncovers discrepancies in behavioral fidelity between teacher and student models that are often missed by traditional accuracy-based evaluations. As future work, we plan to integrate *MetaCompress* into the knowledge distillation pipeline to enable feedback-driven optimization of the distillation process, aiming to reduce the knowledge and representational gaps in student models and ensure behavioral fidelity with their teacher counterparts.

Data Availability

Our code and the corresponding dataset are publicly available to enhance further research[11].

Acknowledgments

This research is supported in part by the Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery Grants program, the Canada Foundation for Innovation's John R. Evans Leaders Fund (CFI-JELF), and by the industry-stream NSERC CREATE in Software Analytics Research (SOAR).

Declaration of Generative AI and AI-assisted tools

During the preparation of this work, the author(s) used Grammarly [85] and ChatGPT [86] to find grammatical mistakes and improve sentence clarity/ presentation. After using these tools/services, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

[11] `https://doi.org/10.5281/zenodo.16127320`

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Advances in neural information processing systems 30 (2017).

[2] S. Arshad, S. Abid, S. Shamail, Codebert for code clone detection: A replication study, in: 16th IWSC, IEEE, 2022, pp. 39–45.

[3] T. Ahmed, K. S. Pai, P. Devanbu, E. Barr, Automatic semantic augmentation of language model prompts (for code summarization), in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13.

[4] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, Y. Chen, Vulnerability detection with code language models: How far are we?, arXiv preprint arXiv:2403.18624 (2024).

[5] J. Chen, X. Hu, Z. Li, C. Gao, X. Xia, D. Lo, Code search is all you need? improving code suggestions with code search, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13.

[6] J. Shi, Z. Yang, B. Xu, H. J. Kang, D. Lo, Compressing pre-trained models of code into 3 mb, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, ASE '22, Association for Computing Machinery, New York, NY, USA, 2023. doi:10.1145/3551349.3556964.
URL https://doi.org/10.1145/3551349.3556964

[7] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green ai, Communications of the ACM 63 (12) (2020) 54–63.

[8] V. J. Hellendoorn, S. Proksch, H. C. Gall, A. Bacchelli, When code completion fails: A case study on real-world completions, in: 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), IEEE, 2019, pp. 960–970.

[9] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, arXiv preprint arXiv:1503.02531 (2015).

[10] J. Shi, Z. Yang, H. J. Kang, B. Xu, J. He, D. Lo, Greening large language models of code, in: Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society, ICSE-SEIS'24, Association for Computing Machinery, New York, NY, USA, 2024, p. 142–153. doi:10.1145/3639475.3640097.
URL https://doi-org.libproxy.smu.edu.sg/10.1145/3639475.3640097

[11] J. Shi, Z. Yang, D. Lo, Efficient and green large language models for software engineering: Vision and the road ahead, ACM Transactions on Software Engineering and Methodology (2024).

[12] X. Wei, S. K. Gonugondla, S. Wang, W. Ahmad, B. Ray, H. Qian, X. Li, V. Kumar, Z. Wang, Y. Tian, et al., Towards greener yet powerful code generation via quantization: An empirical study, in: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2023, pp. 224–236.

[13] M. Saad, J. A. H. López, B. Chen, D. Varró, T. Sharma, Alpine: An adaptive language-agnostic pruning method for language models for code, arXiv preprint arXiv:2407.04147 (2024).

[14] G. d'Aloisio, L. Traini, F. Sarro, A. Di Marco, On the compression of language models for code: An empirical study on codebert, arXiv preprint arXiv:2412.13737 (2024).

[15] Y. Chen, Y. Ye, Z. Li, Y. Ma, C. Gao, Smaller but better: Self-paced knowledge distillation for lightweight yet effective lcms, Proceedings of the ACM on Software Engineering 2 (FSE) (2025) 3057–3080.

[16] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, et al., Codebert: A pre-trained model for programming and natural languages, arXiv:2002.08155 (2020).

[17] Z. Yang, J. Shi, J. He, D. Lo, Natural attack for pre-trained models of code, in: Proceedings of the 44th ICSE, 2022, pp. 1482–1493.

[18] H. Zhang, Z. Li, G. Li, L. Ma, Y. Liu, Z. Jin, Generating adversarial examples for holding robustness of source code processing models, in: Proceedings of the AAAI Conference on AI, Vol. 34, 2020, pp. 1169–1176.

[19] Z. Zeng, H. Tan, H. Zhang, J. Li, Y. Zhang, L. Zhang, An extensive study on pre-trained models for program understanding and generation, in: 31st ACM SIGSOFT ISSTA, 2022, pp. 39–51.

[20] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. Tse, Z. Q. Zhou, Metamorphic testing: A review of challenges and opportunities, ACM Computing Surveys (CSUR) 51 (1) (2018) 1–27.

[21] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, et al., Graphcodebert: Pre-training code representations with data flow, arXiv:2009.08366 (2020).

[22] A. Panichella, Metamorphic-based many-objective distillation of llms for code-related tasks, in: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), IEEE Computer Society, 2025, pp. 766–766.

[23] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang, et al., Codexglue: A machine learning benchmark dataset for code understanding and generation, arXiv:2102.04664 (2021).

[24] W. U. Ahmad, S. Chakraborty, B. Ray, K.-W. Chang, Unified pre-training for program understanding and generation, arXiv:2103.06333 (2021).

[25] Y. Wang, W. Wang, S. Joty, S. C. Hoi, Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, arXiv:2109.00859 (2021).

[26] J. Devlin, Bert: Pre-training of deep bidirectional transformers for language understanding, arXiv:1810.04805 (2018).

[27] Y. Liu, Roberta: A robustly optimized bert pretraining approach, arXiv:1907.11692 364 (2019).

[28] V. Sanh, Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter, arXiv:1910.01108 (2019).

[29] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, M. Brockschmidt, Codesearchnet challenge: Evaluating the state of semantic code search, arXiv:1909.09436 (2019).

[30] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, H. Wang, Large language models for software engineering: A systematic literature review, ACM Transactions on Software Engineering and Methodology 33 (8) (2024) 1–79.

[31] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei, et al., Starcoder 2 and the stack v2: The next generation, arXiv preprint arXiv:2402.19173 (2024).

[32] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, arXiv preprint arXiv:2107.03374 (2021).

[33] X. Liu, X. Liu, L. Bo, X. Wu, Y. Yang, X. Sun, F. Zhou, Pioneer: improving the robustness of student models when compressing pre-trained models of code, Automated Software Engineering 33 (1) (2026) 1–30.

[34] C. Xu, J. McAuley, A survey on model compression and acceleration for pretrained language models, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 10566–10575.

[35] V. Sanh, T. Wolf, A. Rush, Movement pruning: Adaptive sparsity by fine-tuning, Advances in neural information processing systems 33 (2020) 20378–20389.

[36] O. Zafrir, G. Boudoukh, P. Izsak, M. Wasserblat, Q8bert: Quantized 8bit bert, in: 2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing-NeurIPS Edition (EMC2-NIPS), IEEE, 2019, pp. 36–39.

[37] T. Elsken, J. H. Metzen, F. Hutter, Neural architecture search: A survey, Journal of Machine Learning Research 20 (55) (2019) 1–21.

[38] W. Chen, J. Wilson, S. Tyree, K. Weinberger, Y. Chen, Compressing neural networks with the hashing trick, in: International conference on machine learning, PMLR, 2015, pp. 2285–2294.

[39] T. N. Sainath, B. Kingsbury, V. Sindhwani, E. Arisoy, B. Ramabhadran, Low-rank matrix factorization for deep neural network training with high-dimensional output targets, in: 2013 IEEE international conference on acoustics, speech and signal processing, IEEE, 2013, pp. 6655–6659.

[40] S. Afrin, B. Xu, A. Mastropaolo, Is quantization a deal-breaker? empirical insights from large code models, arXiv preprint arXiv:2507.09665 (2025).

[41] J. Zhang, J. Chen, D. Hao, Y. Xiong, B. Xie, L. Zhang, H. Mei, Search-based inference of polynomial metamorphic relations, in: Proceedings of the 29th ACM/IEEE international conference on Automated software engineering, 2014, pp. 701–712.

[42] B. Yang, M. A. A. Mamun, J. M. Zhang, G. Uddin, Hallucination detection in large language models with metamorphic relations, arXiv preprint arXiv:2502.15844 (2025).

[43] X. Xie, J. W. Ho, C. Murphy, G. Kaiser, B. Xu, T. Y. Chen, Testing and validating machine learning classifiers by metamorphic testing, Journal of Systems and Software 84 (4) (2011) 544–558.

[44] T. Y. Chen, S. C. Cheung, S. M. Yiu, Metamorphic testing: a new approach for generating next test cases, arXiv preprint arXiv:2002.12543 (2020).

[45] D. Xiao, Z. Liu, Y. Yuan, Q. Pang, S. Wang, Metamorphic testing of deep learning compilers, Proceedings of the ACM on Measurement and Analysis of Computing Systems 6 (1) (2022) 1–28.

[46] S. Segura, J. A. Parejo, J. Troya, A. Ruiz-Cortés, Metamorphic testing of restful web apis, in: Proceedings of the 40th International Conference on Software Engineering, 2018, pp. 882–882.

[47] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, R. Fergus, Intriguing properties of neural networks, arXiv:1312.6199 (2013).

[48] H. Zhang, Z. Fu, G. Li, L. Ma, Z. Zhao, H. Yang, Y. Sun, Y. Liu, Z. Jin, Towards robustness of deep program processing models—detection, estimation, and enhancement, ACM Transactions on Software Engineering and Methodology (TOSEM) 31 (3) (2022) 1–40.

[49] S. Kullback, R. A. Leibler, On information and sufficiency, The annals of mathematical statistics 22 (1) (1951) 79–86.

[50] J. Svajlenko, J. F. Islam, I. Keivanloo, C. K. Roy, M. M. Mia, Towards a big data curated benchmark of inter-project code clones, in: 2014 IEEE International Conference on Software Maintenance and Evolution, IEEE, 2014, pp. 476–480.

[51] W. Wang, G. Li, B. Ma, X. Xia, Z. Jin, Detecting code clones with graph neural network and flow-augmented abstract syntax tree, in: 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2020, pp. 261–271.

[52] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, Advances in NIPS 32 (2019).

[53] X. Du, M. Wen, Z. Wei, S. Wang, H. Jin, An extensive study on adversarial attack against pre-trained models of code, in: 31st FSE, 2023, pp. 489–501.

[54] M. Friedman, The use of ranks to avoid the assumption of normality implicit in the analysis of variance, Journal of the american statistical association 32 (200) (1937) 675–701.

[55] R. F. Woolson, Wilcoxon signed-rank test, Wiley encyclopedia of clinical trials (2007) 1–3.

[56] W. E. Wong, J. R. Horgan, S. London, H. Agrawal, A study of effective regression testing in practice, in: PROCEEDINGS The Eighth International Symposium On Software Reliability Engineering, IEEE, 1997, pp. 264–274.

[57] Y. Jia, M. Harman, An analysis and survey of the development of mutation testing, IEEE transactions on software engineering 37 (5) (2010) 649–678.

[58] X. Xie, Z. Zhang, T. Y. Chen, Y. Liu, P.-L. Poon, B. Xu, Mettle: A metamorphic testing approach to assessing and validating unsupervised machine learning systems, IEEE Transactions on Reliability 69 (4) (2020) 1293–1322.

[59] Z. Sun, Z. Chen, J. Zhang, D. Hao, Fairness testing of machine translation systems, ACM Transactions on Software Engineering and Methodology 33 (6) (2024) 1–27.

[60] W.-C. Kwan, X. Zeng, Y. Jiang, Y. Wang, L. Li, L. Shang, X. Jiang, Q. Liu, K.-F. Wong, Mt-eval: A multi-turn capabilities evaluation benchmark for large language models, arXiv preprint arXiv:2401.16745 (2024).

[61] J. M. Zhang, M. Harman, L. Ma, Y. Liu, Machine learning testing: Survey, landscapes and horizons, IEEE Transactions on Software Engineering 48 (1) (2020) 1–36.

[62] W. Wu, Y. Cao, N. Yi, R. Ou, Z. Zheng, Detecting and reducing the factual hallucinations of large language models with metamorphic testing, Proceedings of the ACM on Software Engineering 2 (FSE) (2025) 1432–1453.

[63] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, nature 521 (7553) (2015) 436–444.

[64] Y. Shang, G. Liu, R. R. Kompella, Y. Yan, Enhancing post-training quantization calibration through contrastive learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 15921–15930.

[65] Y. Ji, Y. Xiang, J. Li, Q. Xia, P. Li, X. Duan, Z. Wang, M. Zhang, Beware of calibration data for pruning large language models, arXiv preprint arXiv:2410.17711 (2024).

[66] J. Ba, R. Caruana, Do deep nets really need to be deep?, Advances in neural information processing systems 27 (2014).

[67] L. Wang, K.-J. Yoon, Knowledge distillation and student-teacher learning for visual intelligence: A review and new outlooks, IEEE transactions on pattern analysis and machine intelligence 44 (6) (2021) 3048–3068.

[68] R. Wang, Z. Yang, C. Gao, X. Xia, Q. Liao, An empirical study of knowledge distillation for code understanding tasks, arXiv preprint arXiv:2508.15423 (2025).

[69] X. Zhu, J. Li, Y. Liu, C. Ma, W. Wang, A survey on model compression for large language models, Transactions of the Association for Computational Linguistics 12 (2024) 1556–1577.

[70] M. Xu, D. Cai, W. Yin, S. Wang, X. Jin, X. Liu, Resource-efficient algorithms and systems of foundation models: A survey, ACM Computing Surveys 57 (5) (2025) 1–39.

[71] J. Xu, W. Zhou, Z. Fu, H. Zhou, L. Li, A survey on green deep learning, arXiv preprint arXiv:2111.05193 (2021).

[72] M. A. Gordon, K. Duh, N. Andrews, Compressing bert: Studying the effects of weight pruning on transfer learning, arXiv preprint arXiv:2002.08307 (2020).

[73] C. Xu, W. Zhou, T. Ge, K. Xu, J. McAuley, F. Wei, Beyond preserved accuracy: Evaluating loyalty and robustness of bert compression, arXiv preprint arXiv:2109.03228 (2021).

[74] S. Ye, K. Xu, S. Liu, H. Cheng, J.-H. Lambrechts, H. Zhang, A. Zhou, K. Ma, Y. Wang, X. Lin, Adversarial robustness vs. model compression, or both?, in: Proceedings of the IEEE/CVF international conference on computer vision, 2019, pp. 111–120.

[75] M. Du, S. Mukherjee, Y. Cheng, M. Shokouhi, X. Hu, A. H. Awadallah, Robustness challenges in model distillation and pruning for natural language understanding, arXiv preprint arXiv:2110.08419 (2021).

[76] S. K. Gourtani, N. Meratnia, Improving robustness of compressed models with weight sharing through knowledge distillation, in: 2024 IEEE 10th International Conference on Edge Computing and Scalable Cloud (EdgeCom), IEEE, 2024, pp. 13–21.

[77] J. Zhu, L. Wang, X. Han, Safety and performance, why not both? bi-objective optimized model compression toward ai software deployment, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–13.

[78] S. Sun, Y. Cheng, Z. Gan, J. Liu, Patient knowledge distillation for bert model compression, arXiv preprint arXiv:1908.09355 (2019).

[79] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, Q. Liu, Tinybert: Distilling bert for natural language understanding, arXiv preprint arXiv:1909.10351 (2019).

[80] C. Buciluǎ, R. Caruana, A. Niculescu-Mizil, Model compression, in: Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining, 2006, pp. 535–541.

[81] Y. Jiang, C. Chan, M. Chen, W. Wang, Lion: Adversarial distillation of proprietary large language models, arXiv preprint arXiv:2305.12870 (2023).

[82] T. Zhang, S. Ye, K. Zhang, X. Ma, N. Liu, L. Zhang, J. Tang, K. Ma, X. Lin, M. Fardad, et al., Structadmm: A systematic, high-efficiency framework of structured weight pruning for dnns, arXiv preprint arXiv:1807.11091 (2018).

[83] R. Tang, Y. Lu, L. Liu, L. Mou, O. Vechtomova, J. Lin, Distilling task-specific knowledge from bert into simple neural networks, arXiv preprint arXiv:1903.12136 (2019).

[84] C. Xu, W. Zhou, T. Ge, F. Wei, M. Zhou, Bert-of-theseus: Compressing bert by progressive module replacing, arXiv preprint arXiv:2002.02925 (2020).

[85] G. Inc., Grammarly, [Online writing assistant] (2025).
URL https://www.grammarly.com

[86] OpenAI, Chatgpt, [Large language model] (2025).
URL https://chatgpt.com